

In-Class Assignment: RabbitMQ

Asynchronous Communication using Message Queues

Objective

The objective of this assignment is to understand simple asynchronous communication using RabbitMQ message queues and exchanges.

In this assignment, students need to implement **1-to-N communication** using a Fanout exchange with two independent consumers. For your reference, **1-to-1 communication** is already implemented.

The assignment focuses on:

- Understanding producer, queue, exchange, and consumer roles.
- Observing 1-to-1 communication.
- Implementing 1-to-N communication using a Fanout exchange.
- Processing the same message differently in two consumers.
- Observing consumer behavior when one consumer is stopped and restarted.

1. Files Provided

The assignment contains the following files.

RabbitMQ Sample Files

The `rabbitmq/` directory contains examples demonstrating different RabbitMQ communication mechanisms:

```
send.py
receive.py

send_logs_direct.py
receive_logs_direct.py

send_logs_topic.py
receive_logs_topic.py

send_logs_fanout.py
receive_logs_fanout.py
```

The Basic, Direct, Topic, and Fanout examples are provided for reference and learning.

1-1 Demonstration Files

The following files are provided for the 1-to-1 demonstration:

```
modified_send.py
modified_receive.py
messages.txt
```

Input File

The file `messages.txt` contains one message per line.

For example:

```
RabbitMQ makes asynchronous communication simple
Students are learning distributed systems
The server received a new request
Message queues help decouple applications
Python is useful for building network applications
Twinkle twinkle little star Mahesh Babu Superstar
Thala for a reason
```

Each line is treated as one message.

Fanout Exercise Files

The following files are provided as the starting point for the student exercise:

```
send_logs_fanout.py
receive_logs_fanout.py
messages.txt
```

Students will modify/create the required Fanout producer and consumer programs.

2. Installing and Starting RabbitMQ

Make sure Docker is installed and running.

Start a RabbitMQ container using:

```
docker run -d --hostname rabbit-host --name rabbitmq \
-p 5672:5672 -p 15672:15672 \
rabbitmq:3-management
```

The ports are used as follows:

- 5672: AMQP port used by Python RabbitMQ clients.
- 15672: RabbitMQ management interface.

The RabbitMQ management interface can be accessed at:

```
http://localhost:15672
```

If RabbitMQ is already running, do not start another RabbitMQ container.

Optional: Installing RabbitMQ directly on macOS

macOS users may alternatively install RabbitMQ using Homebrew:

```
brew update
brew install rabbitmq
brew services start rabbitmq
```

Check the RabbitMQ status using:

```
rabbitmqctl status
```

3. Installing the Python RabbitMQ Library

The Python programs use the `pika` library.

Install it using:

```
pip install pika
```

If you are using a Python virtual environment, activate the environment before installing `pika` and running the programs.

4. Part A – 1-to-1 Communication Demonstration

1-to-1 Implementation

The files used for the 1-to-1 demonstration are:

```
modified_send.py
modified_receive.py
messages.txt
```

Here, `send.py` and `receive.py` from the provided RabbitMQ sample have been modified for this assignment.

- `modified_send.py` is the modified producer. It reads the input file `messages.txt` and publishes each non-empty line as a message to the RabbitMQ queue.
- `modified_receive.py` is the modified consumer. It receives each message from the RabbitMQ queue and calculates the number of words in the message.

The purpose of these modifications is to demonstrate a simple 1-to-1 asynchronous communication system where messages are read from an input file, sent through a RabbitMQ queue, and processed by a single consumer.

Running the 1-to-1 Demonstration

Open two terminals.

Terminal 1 – Start the Consumer

```
python3 modified_receive.py
```

The consumer waits for messages.

Terminal 2 – Start the Producer

```
python3 modified_send.py messages.txt
```

The producer reads the messages from `messages.txt` and sends them to the RabbitMQ queue.

The consumer receives the messages and displays the corresponding word count.

5. Part B – Student Exercise: Fanout 1-to-N

Objective of the Fanout Exercise

Students need to implement a 1-to-N communication system using a RabbitMQ Fanout exchange. The same message published to the Fanout exchange should be delivered to both consumers.

Fanout Producer

Start with the provided:

```
send_logs_fanout.py
```

Modify the producer so that it:

1. Accepts the input filename as a command-line argument.
2. Reads the file line by line.
3. Treats each non-empty line as one message.
4. Publishes each message to the Fanout exchange.

The producer should be run using:

```
python3 modified_send_logs_fanout.py messages.txt
```

Word Consumer and Vowel Consumer

Use the provided:

```
receive_logs_fanout.py
```

as the reference implementation for both consumers.

You must create two modified consumer programs:

```
word_consumer.py  
vowel_consumer.py
```

Both programs should follow the same RabbitMQ/Fanout structure as `receive_logs_fanout.py`. In particular, each consumer should:

1. Connect to RabbitMQ.

2. Declare and use the Fanout exchange.
3. Create its own temporary queue.
4. Bind the queue to the Fanout exchange.
5. Receive messages from its queue.

The only difference between the two consumers is the processing performed on each received message.

Word Consumer

Modify the consumer (`receive_logs_fanout.py`) so that it:

- is saved as `word_consumer.py`;
- **calculates the number of words in each received message**; and
- displays the message and its word count.

Vowel Consumer

Modify the consumer (`receive_logs_fanout.py`) so that it:

- is saved as `vowel_consumer.py`;
- **calculates the number of vowels in each received message**; and
- displays the message and its vowel count.

The two consumers must use **separate queues**. Since both queues are bound to the same Fanout exchange, each consumer should receive its own copy of every message published to the exchange.

Running the Fanout Implementation

Run the two consumers before starting the producer.

Terminal 1 – Word Consumer

```
python word_consumer.py
```

Terminal 2 – Vowel Consumer

```
python vowel_consumer.py
```

Terminal 3 – Producer

```
python send_logs_fanout.py messages.txt
```

Observe that the same messages are received by both consumers.

Verifying the Fanout Communication

To verify that the Fanout implementation is working correctly:

1. Make sure both `word_consumer.py` and `vowel_consumer.py` are running before starting the producer.
2. Run the producer:

```
python3 send_logs_fanout.py messages.txt
```

3. Check the output of both consumers.
4. The same messages from `messages.txt` should appear in **both** consumers.
5. The Word Consumer should display the word count for each message, while the Vowel Consumer should display the vowel count for the same message.

If every message is received by both consumers and each consumer performs its respective processing, the Fanout 1-to-N communication is working correctly.

6. Part C – Consumer Failure Experiment

Experiment Procedure

After completing the Fanout implementation, perform the following experiment.

1. Start both `word_consumer.py` and `vowel_consumer.py`.
2. Stop one of the consumers using `Ctrl+C`.
3. Run the producer:

```
python3 send_logs_fanout.py messages.txt
```

4. Observe the consumer that is still running.
5. Restart the consumer that was stopped.
6. Run the producer again:

```
python3 send_logs_fanout.py messages.txt
```

7. Observe the behavior of both consumers.

Write-Down Questions

Create an `answers.txt` file containing the following questions and your answers.

1. **Which consumer did you stop?** Mention the consumer filename.
2. **While the selected consumer was stopped, did the other consumer continue to receive messages?** Answer **Yes/No**.
3. **After restarting the stopped consumer, did it receive the messages that were sent while it was stopped?** Answer **Yes/No** and briefly explain why.
4. **After restarting the stopped consumer, run the producer again. Did both consumers receive the new messages?** Answer **Yes/No** and briefly explain your observation.

7. Submission Requirements

Submit the following files on LMS.

```
modified_send_logs_fanout.py  
word_consumer.py  
vowel_consumer.py  
answers.txt
```